

# PPL Week-9

Rachit Takate – 230962294 – B54

**I. Write a program in CUDA to perform parallel Sparse Matrix - Vector multiplication using compressed sparse row (CSR) storage format. Represent the input sparse matrix in CSR format in the host code.**

```
#include <cuda.h>
#include <stdio.h>

#define M 3
#define N 3

__global__ void mulCSR(float *matData, float *vec, float *res, uint *cols, uint
*rows, uint rLen) {
    uint i = blockDim.x * blockIdx.x + threadIdx.x;

    if(i < rLen - 1) {
        float acc = 0.0f;
        for(uint k = rows[i]; k < rows[i + 1]; k++)
            acc += matData[k] * vec[cols[k]];

        res[i] = acc;
    }
}

void printArrf(float *arr, unsigned int len) {
    for(unsigned int i = 0; i < len; i++)
        printf("%.0f ", arr[i]);
    printf("\n");
}

int main() {
    float matData[] = {1, 2, 3, 4, 5, 6},
    vec[] = {2, 3, 4},
    res[N];
    uint cols[] = {0, 2, 0, 1, 2, 2},
    rows[] = {0, 2, 5, 6};

    float *dMatData, *dVec, *dRes;
    uint *dCols, *dRows;

    size_t sData = sizeof(float),
    sVec = sData * N,
    sCols = sizeof(int),
    sRows = sCols * 4;
    sData *= 6;
    sCols *= 6;

    cudaMalloc((void **) &dMatData, sData);
    cudaMalloc((void **) &dVec, sVec);
    cudaMalloc((void **) &dRes, sVec);
    cudaMalloc((void **) &dCols, sData);
    cudaMalloc((void **) &dRows, sCols);

    cudaMemcpy(dMatData, matData, sData, cudaMemcpyHostToDevice);
    cudaMemcpy(dVec, vec, sVec, cudaMemcpyHostToDevice);
    cudaMemcpy(dCols, cols, sCols, cudaMemcpyHostToDevice);
    cudaMemcpy(dRows, rows, sRows, cudaMemcpyHostToDevice);
```

```

mulCSR<<<1, M>>>(dMatData, dVec, dRes, dCols, dRows, 4);
cudaMemcpy(res, dRes, sVec, cudaMemcpyDeviceToHost);
printf("Result: ");
printArrf((float *) res, N);

cudaFree(dMatData);
cudaFree(dVec);
cudaFree(dRes);
cudaFree(dCols);
cudaFree(dRows);

printf("Rachit 54\n");
return 0;
}

```

## Output

```

(base) mca@computinglab26-26:~/230962294/PP Lab/Week9$ nvcc Q1.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week9$ ./a.out
Result: 10 38 24
Rachit 54

```

**II. Write a program in CUDA to read  $M \times N$  matrix A and replace 1st row of this matrix by same elements, 2nd row elements by square of each element and 3rd row elements by cube of each element and so on.**

```

#include <cuda.h>
#include <stdio.h>

#define M 3
#define N 4

__global__ void powRow(float *aMat, uint m, uint n) {
    uint i = blockDim.x * blockIdx.x + threadIdx.x,
        j = blockDim.y * blockIdx.y + threadIdx.y;

    if(i < m && j < n)
        aMat[i * n + j] = powf(aMat[i * n + j], i + 1);
}

void printMatf(float *mat, uint m, uint n) {
    for(uint i = 0; i < m; i++) {
        for(uint j = 0; j < n; j++)
            printf("%.0f ", mat[i * n + j]);
        printf("\n");
    }
}

int main() {
    float matA[][N] = {
        {0, 1, 2, 3},
        {4, 5, 6, 7},
        {8, 9, 10, 11}
    };
    float *dA;

    size_t sA = sizeof(float) * M * N;

    cudaMalloc((void **) &dA, sA);

    cudaMemcpy(dA, matA, sA, cudaMemcpyHostToDevice);

    powRow<<<1, dim3(M, N)>>>(dA, M, N);
}

```

```

    cudaMemcpy(mataA, dA, sA, cudaMemcpyDeviceToHost);
    printf("Result:\n");
    printMatf((float *) mataA, M, N);

    cudaFree(dA);
    printf("Rachit 54\n");
    return 0;
}

```

### Output

```

(base) mca@computinglab26-26:~/230962294/PP Lab/Week9$ nvcc Q2.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week9$ ./a.out
Result:
0 1 2 3
16 25 36 49
512 729 1000 1331
Rachit 54

```

**III. Write a CUDA program that reads a matrix A of size MXN and produces an output matrix B of same size such that it replaces all the non-border elements of A with its equivalent 1's complement and remaining elements same as matrix A.**

```

#include <cuda.h>
#include <stdio.h>

#define M 4
#define N 4

__global__ void onesRow(float *aMat, uint m, uint n) {
    uint i = blockDim.x * blockIdx.x + threadIdx.x,
    j = blockDim.y * blockIdx.y + threadIdx.y;

    if(i * j > 0 && i < m - 1 && j < n - 1) {
        uint p = 1, bin = 0, val = aMat[i * n + j];

        while(val > 0) {
            uint bit = val & 1;
            bin += (bit ? 0 : 1) * p;

            val >>= 1;
            p *= 10;
        }

        aMat[i * n + j] = bin;
    }
}

void printMatf(float *mat, uint m, uint n) {
    for(uint i = 0; i < m; i++) {
        for(uint j = 0; j < n; j++)
            printf("%.0f ", mat[i * n + j]);
        printf("\n");
    }
}

int main() {
    float matA[M][N] = {
        {1, 2, 3, 4},
        {6, 5, 8, 3},
        {2, 4, 10, 1},
        {9, 1, 2, 5}
    };
    float *dA;

```

```

size_t sA = sizeof(float) * M * N;

cudaMalloc((void **) &dA, sA);

cudaMemcpy(dA, matA, sA, cudaMemcpyHostToDevice);

onesRow<<<1, dim3(M, N)>>>(dA, M, N);
cudaMemcpy(matA, dA, sA, cudaMemcpyDeviceToHost);
printf("Result:\n");
printMatf((float *) matA, M, N);

cudaFree(dA);
printf("Rachit 54\n");
return 0;
}

```

## Output

```

(base) mca@computinglab26-26:~/230962294/PP Lab/Week9$ nvcc Q3.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week9$ ./a.out
Result:
1 2 3 4
6 10 11 3
2 11 10 1
9 1 2 5
Rachit 54

```